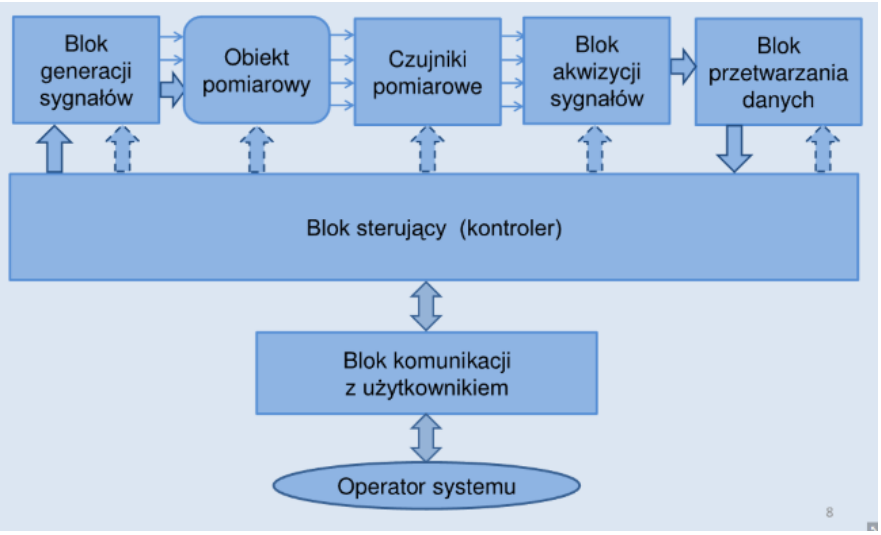
Rozproszone systemy akwizycji - opracowanie (Arnold na ostatnim wykładzie gadał co będzie wymagał - temu podpunkty, zanotowałem tyle ile dałem rady, jak ktoś ma nagranie z ostatniego wykładu gdzie gadał o tych pytaniach to niech dopisze tutaj)

Na zajęcia trzeba wydrukować **listę rozkazów ATmega16**, **tablicę wektorów przerwań**, **wykaz rejestrów**

1. Struktura systemu pomiarowego, jakie funkcje pełnią poszczególne bloki, jakie znaczenie mają kontrolery



Blok generacji sygnałów - wykorzystywany jest do wytwarzania sygnałów wymuszających (programowane źródła napięć i prądów), do generacji sygnałów wzorcowych oraz do wytwarzania sygnałów sterujących elementami wykonawczymi obiektu pomiarowego. Blok ten wymaga jednego lub kilku przetworników C/A w celu wytworzenia sygnałów analogowych.
Składa się z przetwornika cyfrowo analogowego, źródła napięciowe, mogą być dzielniki napięcia i wzmacniacze.
Przykład: na wejście podajemy 1001. Zwraca 9 Voltów.

Obiekt pomiarowy - jego rolą jest to że istnieje i wykonujemy na nim pomiary.

Czujniki pomiarowe - przekształcają wielkości nieelektryczne, lub trudno mierzalne wielkości elektryczne, na łatwo mierzalne wielkości elektryczne, takie jak napięcie stałe, częstotliwość czy przedział czasu. W przypadku środowiska o małych zakłóceniach możemy skorzystać z tanich analogowych czujników. W innym przypadku zalecane są czujniki cyfrowe.

Blok akwizycji sygnałów - pośredniczy między czujnikami pomiarowymi a blokiem przetwarzania danych. Jego zadaniem jest zbieranie sygnałów pomiarowych i przekształcenie ich na postać cyfrową. W bloku akwizycji wykonywana jest wstępna normalizacja sygnału analogowego (często nazywana kondycjonowaniem sygnału).

Arnold mówi: “Jest on najważniejszym blokiem. Jeśli coś mierzymy, to porównujemy to coś z przyjętym wzorcem. To w bloku akwizycji sygnałów dokonuje się właściwy pomiar.”

Blok przetwarzania danych - jest odpowiedzialny za cyfrową obróbkę sygnałów pomiarowych zgodnie z przyjętym algorytmem. W przypadku systemów wymagających przetwarzania w czasie rzeczywistym (real time processing) przyspieszenie obliczeń zapewniają wydzielone bloki funkcjonalne z procesorami sygnałowymi, realizujące złożone i pracochłonne algorytmy przetwarzania danych.
Arnold mówi: to ten blok przekształca kod na dane, którego konwersja wykonuje się według określonego algorytmu. Arytmetyczny blok przetwarzania danych również działa jako bufor, aby kontroler miał czas na przetwarzanie podanych danych.

Kontroler - jest odpowiedzialny za czasowo-przestrzenną koordynację działań systemu, a więc wybór punktów pomiarowych, ustalenie warunków pomiaru, określenie momentu rozpoczęcia pomiaru oraz organizację przepływu informacji. Najczęściej funkcję kontrolera pełnią systemy mikroprocesorowe, które obok czynności sterujących mogą także przetwarzać dane.

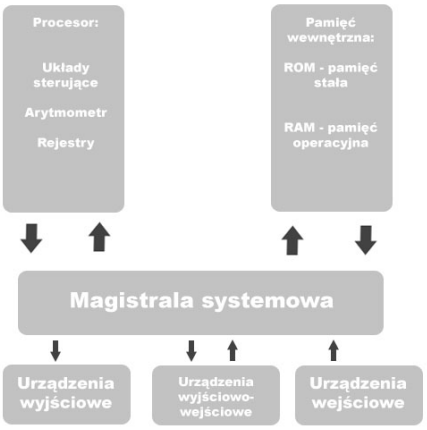
Kontrolerem może być komputer lub mikrokontroler.

Blok komunikacji z użytkownikiem - przeznaczony do wprowadzania i odbierania informacji z systemu przez użytkownika.

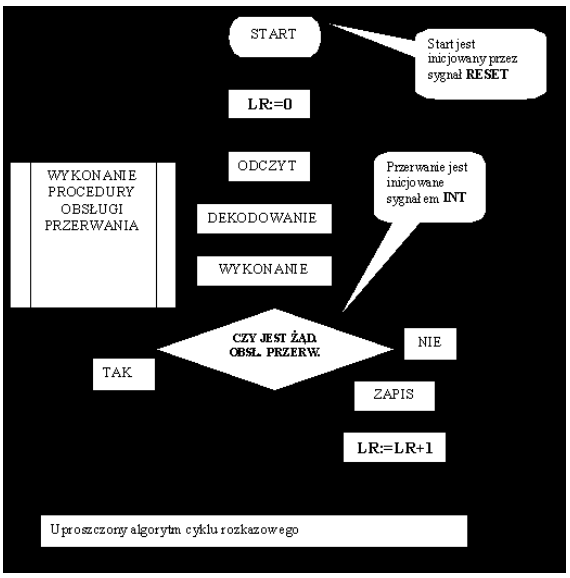
Bloki mogą być realizowane w sposób sprzętowy, sprzętowo-programowy lub programowy. Realizacje sprzętowe są zwykle najdroższe, jednak zapewniają największą szybkość działania. Najtańsze są rozwiązania programowe, jednak szybkość takich rozwiązań jest zwykle mała. Kompromisem cenowo-szybkościowym są rozwiązania sprzętowo programowe, w których część funkcji jest realizowana przez sprzęt, a pozostała część przez program.

2. PC (w sensie komputer) i mikroprocesory jako kontrolery lokalne/kontroler podsystemu stacji nadrzędnej.

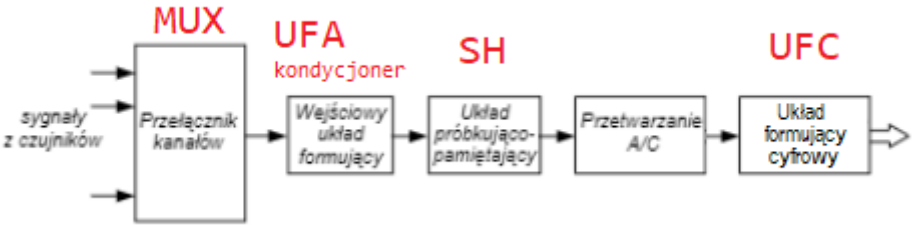
Zajebane z jakiejś strony dla gimnazjalistów



PROCESOR stanowi główny element komputera, ponieważ jest odpowiedzialny za przetwarzanie informacji. Składa się on z układów sterujących, arytmometru oraz rejestrów . Układy sterujące odpowiadają za: dostarczenie arytmometrowi danych do obliczeń z pamięci operacyjnej, przekazywanie wyników obliczeń z powrotem do pamięci oraz właściwą kolejność przetwarzania. W arytmometrze odbywają się wszystkie obliczenia realizowane przez komputer. W rejestrach procesora przechowuje się adresy wybranych miejsc pamięci operacyjnej oraz dane i wyniki obliczeń. W wyróżnionym rejestrze nazywanym licznikiem rozkazów jest umieszczany adres miejsca w pamięci wewnętrznej zawierającego bieżący rozkaz dla procesora. Praca procesora odbywa się w tzw. cyklach rozkazowych. Przebieg jednego cyklu rozkazowego można opisać za pomocą następującego algorytmu: 1. Zawartość miejsca pamięci wewnętrznej wskazywanego przez licznik rozkazów LR zostaje przesłana do układów sterujących procesora, 2. W układach sterujących następuje rozdzielenie otrzymanej informacji na dwa pola: pole operacji i pole argumentów. Pole operacji zawiera adres rozkazu, który należy wykonać. Pole argumentów zawiera adresy, pod którymi są przechowywane dane oraz adres przeznaczenia wyniku. 3. Na podstawie wyznaczonych adresów następuje przesłanie z pamięci wewnętrznej argumentów do odpowiednich rejestrów, a na podstawie adresu rozkazu arytmometr wykonuje odpowiednie działanie (operację arytmetyczną lub logiczną) na zawartościach rejestru. 4. Wynik przetwarzania (wynik wykonanej operacji) jest wysyłany do pamięci wewnętrznej pod adres przeznaczenia wyniku. 5. Następuje zmiana wartości licznika rozkazów LR tak, aby wskazywał on kolejny rozkaz dla procesora. Cykl rozkazowy procesora można opisać również za pomocą narysowanego niżej schematu blokowego:



3. Bloki akwizycji sygnału - schemat wewnętrzny, jakie podzespoły się na to składają, ich funkcje i współpraca.

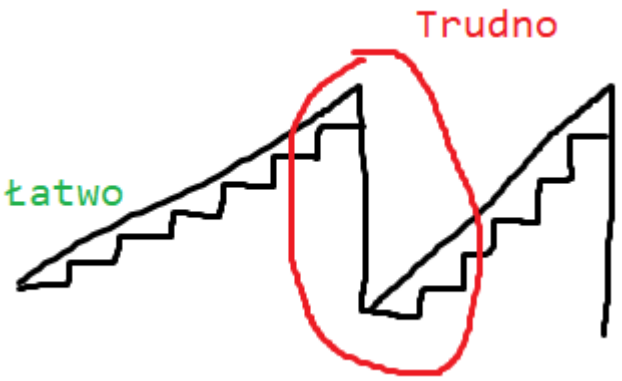


Przełącznik kanałów (MUX) - umożliwia wybór sygnału z poszczególnych czujników według określonego programu. Wykorzystywany jest tutaj multiplekser.

Wejściowy układ formujący analogowy (UFA kondycjoner) - służy do normalizacji sygnału wejściowego, dotyczącej impedancji wejściowej, zakresu i szybkości zmian sygnału, charakterystyki kanału pomiarowego.

Układ próbkująco-pamiętający (SH) - w nim dokonywane jest pobranie próbek wartości sygnału w dyskretnych momentach i zapamiętywanie tych wartości w czasie przetwarzania przetwornika A/C.

W przypadku sygnału piłokształtnego układ będzie znacząco obciążony gdy sygnał spadnie z najwyższej wartości do najniższej.



Sygnał sinusoidalny też będzie miał takie momenty.



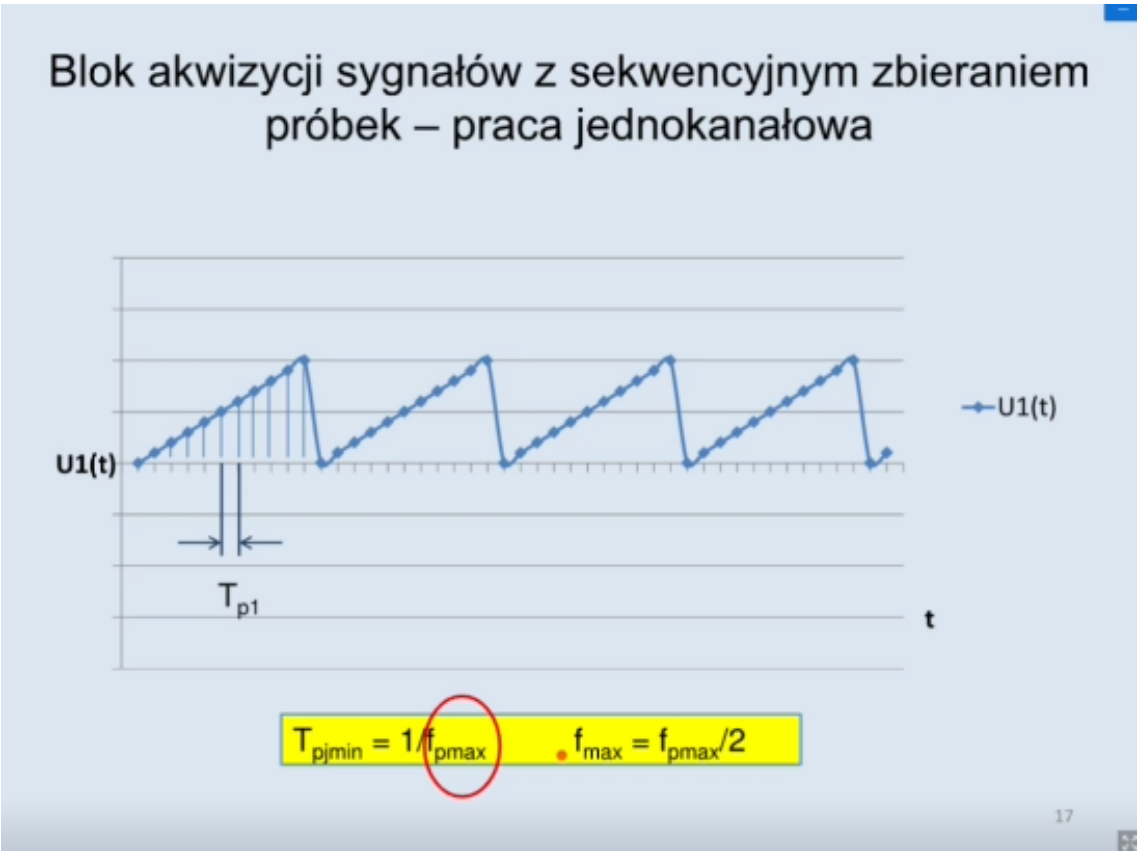
W przypadku pracy wielokanałowej SH w momencie przejścia między kanałowego również będą takie skoki.

Czas akwizycji i pasmo analogowe SH musi zapewnić odporność na takie skoki.

Przetwornik A/C - realizuje konwersję sygnału analogowego na kod cyfrowy, dokładnie to kwantowanie i kodowanie. Przy dużej szybkości zmian sygnału wejściowego stosuje się przetworniki A/C przetwarzające metodą bezpośredniego kodowania lub kompensacyjną, przy małej szybkości metodami integracyjnymi.

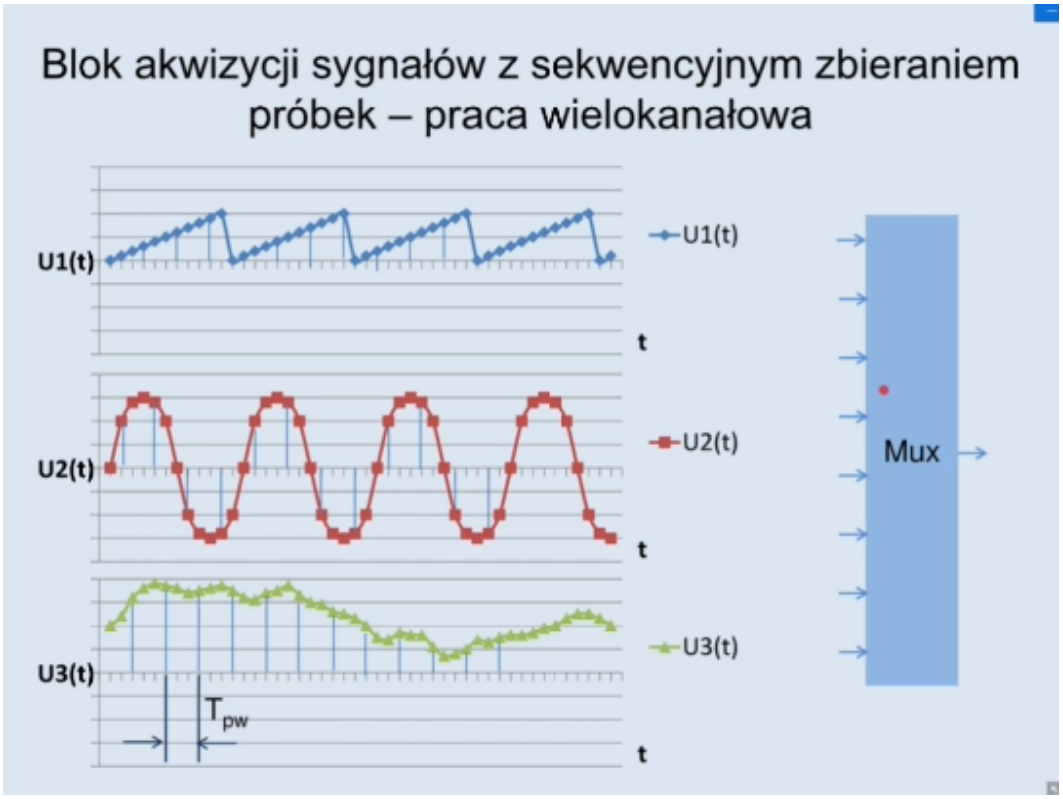
4. Gdzie są stosowane bloki akwizycji sygnału (kiedy bloki z sekwencyjnym zbieraniem próbek, kiedy mieszanym)

Blok akwizycji sygnałów z sekwencyjnym zbieraniem próbek - praca jednokanałowa



Tp1 - okres próbkowania w trybie pracy jednokanałowej
fmax - częstotliwość próbkowania
fpmax - maksymalna częstotliwość próbkowania jaką możemy ustawić na naszym sprzęcie. Jeśli przekroczymy tą wartość, wystąpi aliasing.

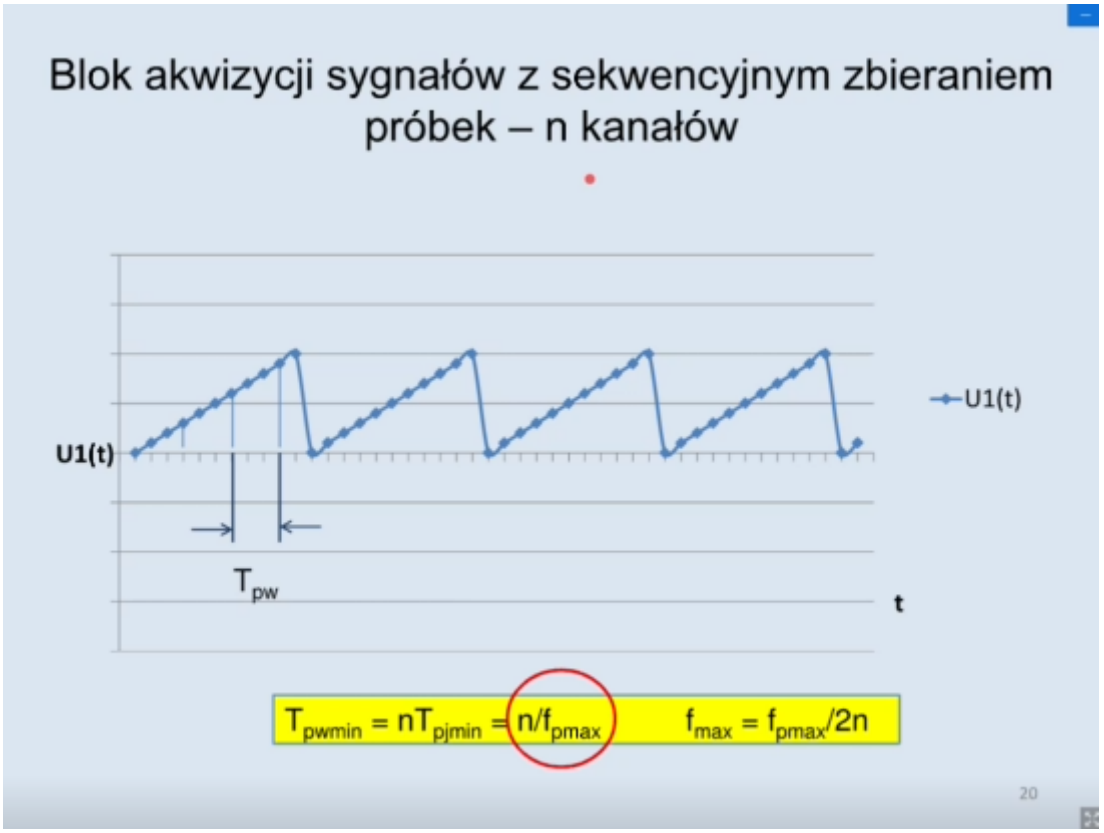
Blok akwizycji sygnałów z sekwencyjnym zbieraniem próbek - praca wielokanałowa



Tpw - okres próbkowania w trybie pracy wielokanałowej
Korzystamy z analogowego multipleksera aby przełączać kanały. Spowolni to proces zbierania próbek. Po każdym pobraniu próbki oprogramowanie powinno przełączyć na następny kanał. Widać to na rysunku - kreski są przesunięte.
Minusy takiego rozwiązania:

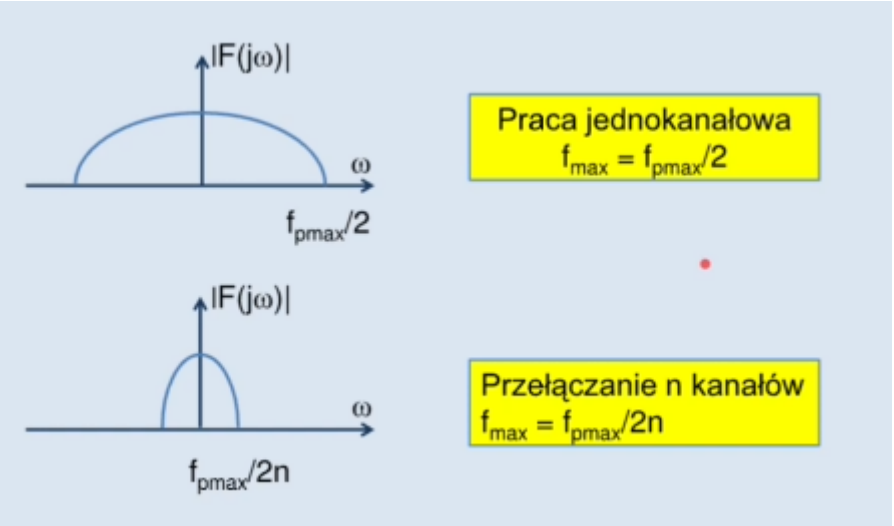
- bardziej złożone sterowanie multiplekserem
- bardziej złożone zbieranie próbek (należy zbierać próbki w pamięci RAM mikrokontrolera). Na jeden kanał jeden licznik adresowy (każdy kanał ma wydzielone komórki pamięci, między którymi się przełączamy przełączając kanały)
- próbkowanie odbywa się z opóźnieniem, np. kanał 3 będzie miał większy odstęp czasu niż w przypadku gdyby był on próbkowany sekwencyjnym próbkowaniem z pracą jednokanałową

Blok akwizycji sygnałów z sekwencyjnym zbieraniem próbek - praca wielokanałowa, n kanałów



Okres próbkowania jest równy n minimalnym okresom z trybu pracy jednokanałowej. Zatem okres to n/fpmax, a nie 1/fpmax.

Maksymalna częstotliwość próbkowania zmniejszyła się n krotnie, pogarszając sytuację i zwiększając wymagania (wymaga maksymalna częstotliwość na naszym sprzęcie wzrasta).



Widmo sygnału próbkowanego znacząco się zmniejsza.

Blok akwizycji z mieszanym zbieraniem próbek



Każdy układ próbkująco-pamiętający widzi tylko swój kanał. Nie musi on skakać między kanałowy jak poprzednio, jest to więc znacząca poprawa.
Przetwornik A/C jest wąskim gardłem, ponieważ musi obsługiwać kilka kanałów na raz.
Mieszane zbieranie próbek ma wielką zaletę - różne kanały są próbkowane jednocześnie.
Niestety, mimo jednoczesnego pobrania n próbek, przez to że przetwornik a/c jest tylko jeden okres próbkowania ciągle jest n krotny w porównaniu do próbkowania jednokanałowego. A/C kolejno jeden po drugim obsługuje kanały.
Podsumowując: Poprawa w warunkach pracy SH, poprawa w korelacji, ale szybkość działania jest taka sama jak w przypadku bloku sekwencyjnego wielokanałowego.

Wzory:

$$T_{pwmin} = nT_{pjmin} = n/f_{pmax} \qquad f_{max} = f_{pmax}/2n$$

n - liczba kanałów
Tpwmin - minimalny okres próbkowania z wieloma kanałami
Tpjmin - minimalny okres próbkowania jednokanałowego
fpmax - maksymalna częstotliwość próbkowania jaką możemy ustawić na naszym sprzęcie.

Problem Drop Rate

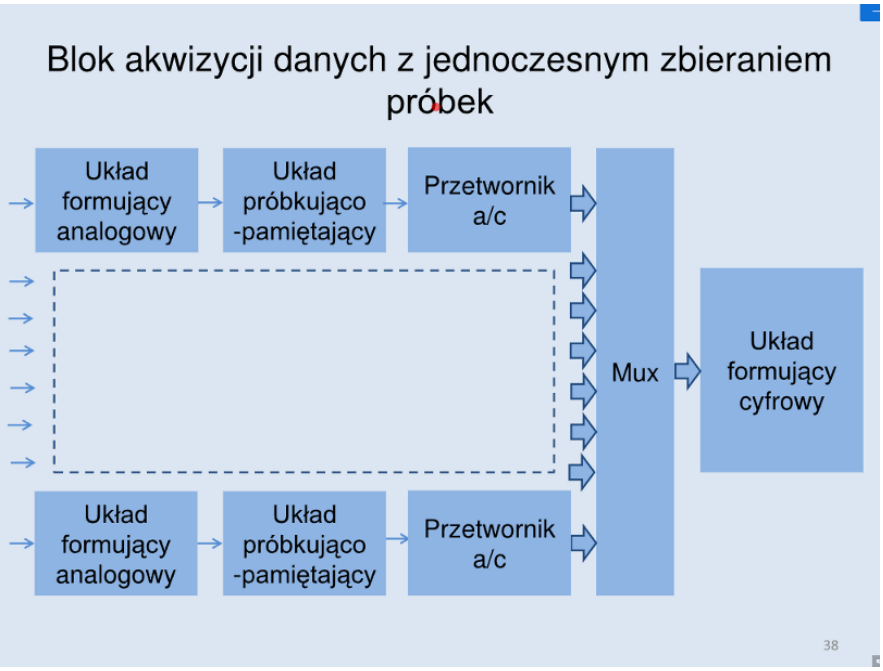
Drop Rate - jak szybko napięcie na kondensatorze spadać lub rosnąć. Jest podawany w [mV/mS]

Problem - Napięcie na kondensatorze nie jest stałe. Kondensator może w sposób niekontrolowany lekko naładować się lub rozładować.

Sposoby poprawy drop rate:

- zastosować szybszy przetwornik A/C
- przesunięcie momentu pobierania próbki, tak jak w przypadku sekwencyjnego zbierania próbek, ale w mniejszym stopniu

Blok akwizycji danych z jednoczesnym zbieraniem próbek



Generalnie mikrokontrolery nie mają wbudowanych kilku przetworników, zatem w tym przypadku przetworniki muszą zostać dokupione oddzielnie.
Próbki sygnału z poszczególnych kanałów zostały przyjęte i przetworzone. Kody czekają na wejściu multipleksera.
Multiplekser ma teraz na wejściu sygnały cyfrowe, zatem mux musi być inny niż w poprzednich przypadkach. Multiplekser będzie układem programowalnym z bramkami logicznymi.
Cyfrowy multiplekser (np. oparty na FPGA) jest o wiele szybszy od multipleksera analogowego przełączającego klucze.
Okres próbkowania wielokanałowego jest równy okresie próbkowania jedno kanałowego, przetwarzanie próbek dzieje się całkowicie in real-time ponieważ MUX nadąża. Uzyskujemy pełną korelację, bloki również mocno zmniejszają problem drop-rate.

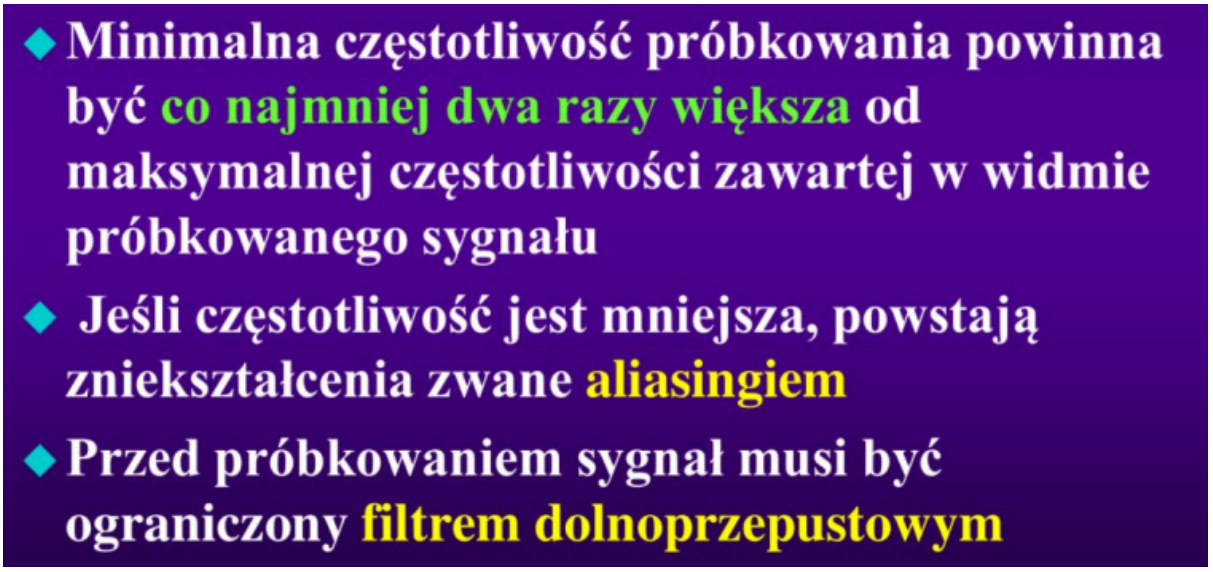
$$T_{pwmin} = T_{pjmin} = 1/f_{pmax}$$

$$f_{max} = f_{pmax}/2$$

Dla najlepszej szybkości polecane są przetworniki z flashami. Rozdzielczość nie jest tak znacząca, jak dobre przetworniki.

Podsumowanie: taka konstrukcja jest droższa, ale o wiele szybsza. Układy próbkująco-pamiętające oraz przetwornik z poprzednich bloków może współpracować z nowym MUXem, jednak wtedy skok prędkości nie będzie tak duży.

5. Twierdzenie Shannona



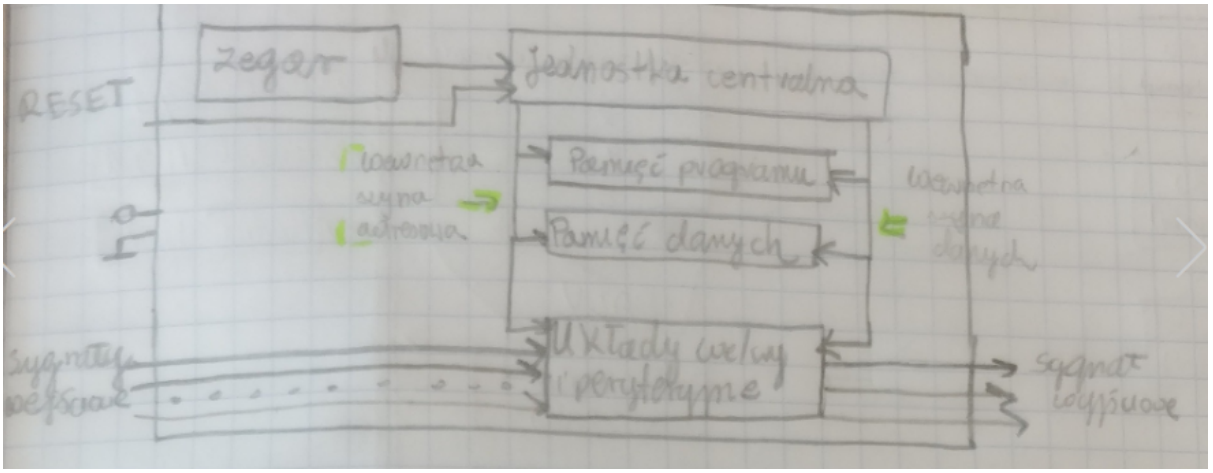
- *Twierdzenie o próbkowaniu* (wersja podstawowa). Jeżeli widmo $X(\omega)$ sygnału $x(t)$ spełnia warunek $X(\omega) \equiv 0$ dla $|\omega| \geq \omega_m$, to sygnał ten można odtworzyć z pełną dokładnością na podstawie jego jego próbek pobieranych z okresem $T_s \leq \pi / \omega_m$.

Próbkowanie (dyskretyzacja) sygnału ciągłego powoduje zwielokrotnienie jego oryginalnego widma w dziedzinie częstotliwości w ten sposób, że obok widma sygnału oryginalnego pojawiają się jego kopie przesunięte o wszystkie całkowite (dodatnie i ujemne) wielokrotności częstotliwości próbkowania, tworząc tzw. obrazy (pierwszy rysunek). Próbkowanie sygnału może wiązać się z jego zniekształceniem wskutek zjawiska aliasingu, czyli nakładania się widm (drugi rysunek). Aby możliwe było wierne odtworzenie sygnału ciągłego, spełnione powinny być przede wszystkim dwa warunki:

- Widmo sygnału oryginalnego oraz kopie tego widma, przesunięte o wszystkie całkowite wielokrotności częstotliwości próbkowania, nie nachodzą na siebie.
- Jest możliwość idealnego odfiltrowania sygnału oryginalnego x(t) z sygnału spróbkowanego x*(t), to znaczy usunięcia zwielokrotnionych kopii X(f) bez zmiany wartości fazy i amplitudy

Jeśli opisane twierdzeniem Kotielnikowa–Shannona warunki nie są spełnione, pojawia się problem aliasingu, którego nie można usunąć bez znajomości oryginalnego sygnału ciągłego.

6. Jaką rolę spełnia mikrokontroler w blokach rozproszonego systemu (tak jak napisałem w punkcie 1, mikrokontroler spełnia zadanie kontrolera *[[[WOW]]]* ale może też przetwarzać dane na wejściu)



Zespoły funkcyjne mikrokontrolera są skupione wokół CPU, która realizuje ciąg operacji zapisany w pamięci programu. Program ten może mieć za zadanie przetworzenie danych zapisanych wcześniej w pamięci danych lub odpowiadać za współpracę z urządzeniami przyłączonymi z zewnątrz.

Dane te są reprezentowane w postaci cyfrowej (ciągi słów dwójkowych). Przesyłanie między jednostką centralną a pamięciami danych programu przez wewnętrzną szynę danych (jest to liczba linii adresowych którymi jednostka centralna adresuje pamięć danych i programu)
Szyna adresowa - jednokierunkowa bo jedynym źródłem adresów jest CPU
szyna danych - dwukierunkowa, bo umożliwia odczyt i zapis danych przez CPU

Jednostka centralna jest synchronicznym układem sekwencyjnym czyli takim w którym zmiany stanów wew. zachodzą przy określonych częstotliwościach impulsów zegara

Najprostsze mikrokontrolery są wyposażone w zespoły linii WE/WY z wewnętrznym buforowaniem, nazywane portami. Każdy port zawiera zwykle 8 linii i służy do równoległej transmisji bajtów danych.

7. Praktyczna część - możliwości przetwarzania danych w ATmega16 - napisz kod np. dodający 8, 16 bitowe próbki.

Przykładowy kod:

STOSOWANIE ROZKAZÓW ADD, ADC, CLR

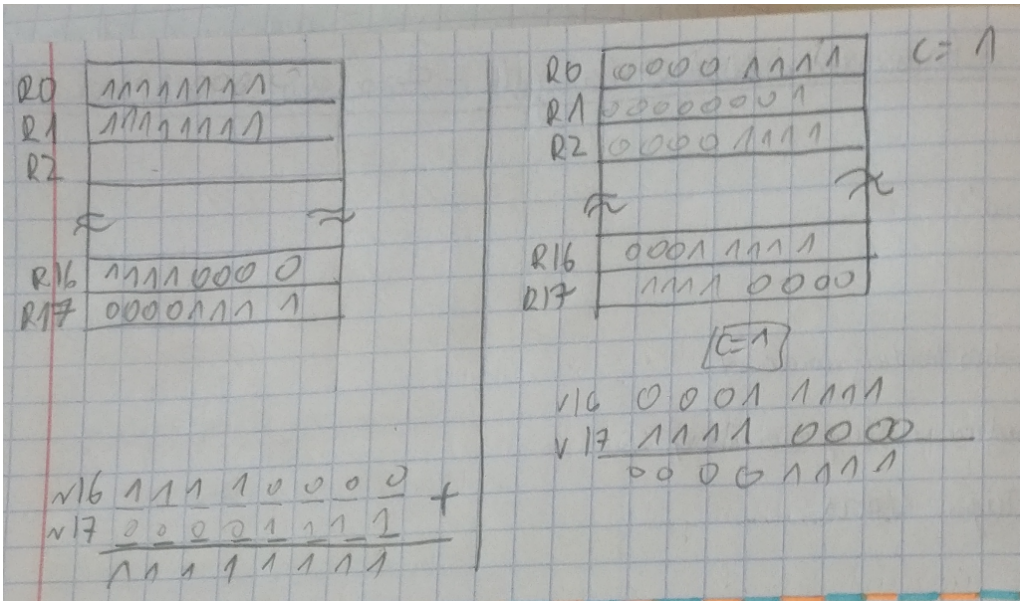
```
;*********************************************
;Dodawanie 2 liczb 8-bitowych bez znaku
;Wynik 8-bitowy
;*********************************************

...
clr r2              ;bajt 0x00
clr r1              ;zeruj MSB wyniku
ldi r16, 0xF0       ;argument 1
ldi r17, 0x0F       ;argument 2
mov r0, r16         ;argument 1 do r0
add r0, r17         ;dodaj argumenty
adc r1, r2          ;dodaj przeniesienie
...
```

```
clr r1 ; czyszczenie r1
clr r2, ; czyszczenie r2

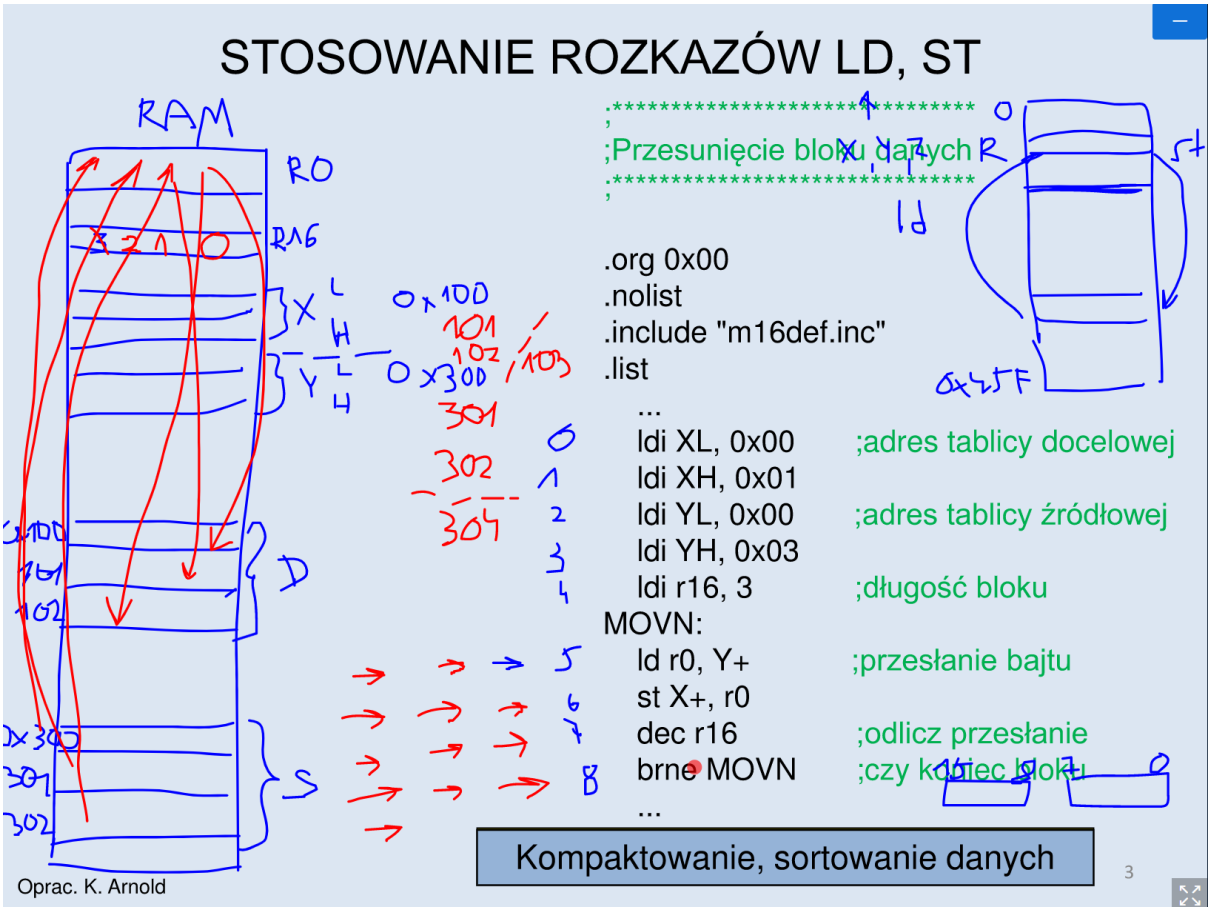
ldi r16, 0xF0 ;wpisanie 240 do r16
ldi r17, 0x0F ;wpisanie 15 do r17

mov r0, r16 ;wpisanie do rejestru r0 wartości z r16
add r0, r17 ;dodaj r0 i r17, wpisz do rejestru r0
adc r1, r2 ;dodaj r1 i r2 z carry biy
```



Dodając 2 liczby 8-bitowe trzeba liczyć się z tym, że otrzymamy wynik 9-bitowy, a mamy 8-bitowy rejestr.
Nie można przenieść wartości tego bitu w carry, bo można przez przypadek go zmodyfikować. Konieczne jest zarezerwowanie dwóch rejestrów, co można wykorzystać na bit znajdujący się w carry

Najpierw czyszczone są rejestry r1, r2 (można też tam wpisać 0, ale jest to nieprofesjonalne). Wprowadzamy argumenty, dodajemy je poprzez ad**c** (c od carry)



ld r0, Y+ -> przesłanie bajtu z Y na r0 i inkrementacja Y

st X+,r0 -> z r0 przenieś wartość na X i inkrementuj X

brne MOVN -> skok do MOVN gdy flaga Z == 1

RJMP - skok o ilość komórek pamięci
JMP - skacze do adresu
IJMP - adres do którego ma zostać wykonany skok jest zapisany w rejestrze Z
.org - pod który adres ma trafić następna dyrektywa

LD - przeniesienie 8 bitowej danej z SRAMu do wskazanego rejestru roboczego. LD wykorzystuje adresy X, Y, Z zawierające informację o adresie którego zawartość chcemy przenieść do rejestru roboczego.

ST - zapamiętaj rejestr roboczy - zapamiętuje wskazane rejestry robocze pod adresami komórek pamięci ukrytych pod X, Y, Z.

Po chłopsku: masz daną komórkę pamięci której dane chcemy przenieść w inne miejsce. Pisząc LD wskazujemy z jakiej komórki chcemy przenieść dane, natomiast ST wskazuje gdzie dane z tej komórki chcemy zapisać.

8. Inicjalizacja stosu, przerwań

Inicjalizacja stosu:

ldi r16, high(RAMEND)
out SPH, r16

ldi r16, low(RAMEND)
out SPL, r16

pod **RAMEND** kryje się ostatni adres statycznego ramu, w naszym przypadku(ATmega16) jest to **0x045F.** Operator high wycina starszą część adresu czyli **04** i wpisujemy to do rejestru 16 bo nie można za pomocą ldi wpisać do SPH dlatego do SPH przesyłamy zawartość r16. Następnie za pomocą operatora low wycinamy młodszą część czyli **5F** i znowu z wykorzystaniem r16 wpisujemy do rejestru **SPL.** Teraz mamy ustawiony stos na najniższą komórkę SRAM, możemy ustawić stos na inną komórkę ale takie ustawienie daje nam najwięcej “miejsca”

Inicjalizacja przerwań:

inicjalizacja przerwań odbywa się poprzez ustawienie odpowiedniej flagi w rejestrze statusowym

7	6	5	4	3	2	1	0
I	T	H	S	V	N	Z	C

SREG

Rejestr statusu- SREG

I- globalne zezwolenie na przerwanie (odmaskowanie ***sei*** I=„1”, zamaskowanie ***cli*** I=„0”, automatycznie odtwarzana po ***reti***)

sei - ustawia flagę I na 1 - przerwania aktywne
cli - ustawia flagę I na 0 - przerwania nieaktywne

9. Mechanizm wywoływania podprogramów z CALL i RET

CALL- zapamiętuje na stosie adres następnej instrukcji programu a następnie przeskakuje do etykiety "etykieta" tak, jak to robiła instrukcja JMP.

RET - po napotkaniu instrukcji RET procesor pobiera ze stosu adres zapamiętany tam przez CALL i przeskakuje pod ten właśnie adres czyli "wraca" do programu nadrzędnego.

10. Przerwania

Przerwania są wykorzystywane do odbioru i wysyłania danych do urządzenia zewnętrznego (dzięki przerwania procesor będzie wiedział że dokładnie w tej chwili czasu mam odebrac/wysłać informacje) Procesor może otrzymać sygnał przerwania z wewn i zewn źródła. Ten mechanizm jest potrzebny po to aby reagować na asynchroniczne zdarzenia.

System przerwań:

- ułatwia mikrokontrolerowi pracę z wieloma układami peryferyjnymi
- pozwala na skrócenie czasu przejścia do podprogramów, których wykonanie jest w danej chwili konieczne
- zwalnia jednostkę centralną od ciągłego sprawdzania warunków od których zależy kolejność wykonywania programu

Aktywność przerwań:

- wszystkie typowe przerwania obsługiwane są pod warunkiem, że zezwolenie globalne jest aktywne (znacznik I=1 rejestru SREG)
- instrukcje sei (I=i) i cli (I=0) ułatwiają zarządzanie zezwoleniem
- po inicjalizacji mikrokontrolera domyślnie blokuje przerwania
- do uaktywnienia konkretnego przerwania konieczne może być odpowiednie ustawienia rejestru sterującego danym modułem

Priorytety i wektory przerwań:

- niebezpieczeństwo: więcej niż jedno żądanie przerwania w danym cyklu zegarowym
- priorytety przerwań - pozwalają ustalić kolejność
- wektor przerwań - adres w pamięci programu pod którym powinien znajdować się podprogram obsługi danego przerwania
- wektory przerwań znajdują się zwykle w początkowej części pamięci flash i zajmują 1 lub 2 słowa rozkazowe
- im wyższy wektor przerwań (dalej w pamięci) tym niższy priorytet

Przerwanie inicjalizacyjne

- jest wykonywane podczas inicjalizacji mikrokontrolera
- ma najwyższy priorytet
- blokada go nie obejmuje
- podprogram inicjalizujący można zacząć od adresu 0x00

Znaczniki przerwań:

- każde źródło przerwania ma dedykowany rejestr w przestrzenia dresowej wejścia-wyjścia, który przechowuje tzw znacznik przerwania
- informuje on o wystąpieniu żądania obsługi przerwania
- w momencie obsługi znacznik jest zerowany
- jeśli przerwania są wyłączone globalnie, to konieczne jest manualne zerowanie znacznika, aby żadna aplikacja go nie wykorzystywała

Obsługa przerwań:

1. Wyłączane jest globalne zezwolenie na przerwania (I=0)
2. Licznik programu zachowywany jest na stosie (4 cykle zegarowe)
3. Wykonywana jest instrukcja umieszczona w wektorze przerwań (zwykle 2,3 cykle zegarowe)

- Całość trwa zwykle 6-7 cykli zegara. W przypadku I=0 obsługa przerwania jest opóźniana tak długo, dopóki I=1
- Powrót z podprogramów odbywa się zwykle przy pomocy reti

11. Mechanizm nadawania i odbioru UART, jego funkcje

Nadawanie: Bity nadawane są z UDR. Trzeba sprawdzić **UDRE** z UCSRA:

Bit 5 **UDRE-** DATA REGISTER EMPTY: informuje o tym, czy rejestr danych UDR jest pusty i możemy wpisać nową daną do wysłania - dzięki temu nie nadpiszemy danych podczas wysyłania UDRE==1 → BUFOR JEST PUSTY
-i skąd będziesz wiedzieć kiedy próbka została dostarczona??RXComplete

***Odbiór*:** UDR jako bufor, sprawdzamy **RXC** z UCSRA czy jest coś do odebrania i należy pamiętać o odblokowaniu **RXEN** i **TXEN** z UCSRB i wybraniu parametrów ramki z UCSRC
UCSRA:
•Bit 7 **RXC**-RECEIVE COMPLETE: Zakończono odbiór znaku RCX==0 →BUFOR ODBIORU JEST PUSTY
UCSRB:
•Bit 4 RXEN- RECEIVER ENABLE
•Bit 3 TXEN- TRANSMITER ENABLE

USART Control and Status Register A-

Bit	7	6	5	4	3	2	1	0	
	RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM	UCSRA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

•Bit 7 RXC-RECEIVE COMPLETE:
Zakończono odbiór znaku
RCX==0 →BUFOR ODBIORU JEST PUSTY

•Bit 6 TXC-TRANSMIT COMPLETE:
Zakończono transmisję całej ramki
TXC==1→BUFOR TRANSMISJI JEST PUSTY

•Bit 5 UDRE- DATA REGISTER EMPTY:
informuje o tym, czy rejestr danych UDR jest pusty i możemy wpisać nową daną do wysłania (jest to sprzętowe rozwiązanie portu szeregowego)- dzięki temu nie nadpiszemy danych podczas wysyłania
UDRE==1 → BUFOR JEST PUSTY

•Bit 4 FE- FRAME ERROR:
Kontrola błędów ramki
FE==1 →kolejny znak w buforze odbioru został odebrany z błędem

•Bit 3 DOR-DATA OVER RUN
DOR==1 →przepełnienie ramki

•Bit 2 PE-PARITY ERROR:
Kontrola Parzystości

•Bit 1 U2X- DOUBLE THE USART TRANSMISSION SPEED
Dwukrotne zwiększenie prędkości transmisji

•Bit 0 MPCM- MULTI-PROCESSOR COMMUNICATION MODE
Wieloprocesorowa komunikacja

USART Control and Status Register B-

Bit	7	6	5	4	3	2	1	0	
	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8	UCSRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

•Bit 7 RXCIE- RX COMPLETE INTERRUPT ENABLE
•Bit 6 TXCIE- TX COMPLETE INTERRUPT ENABLE
•Bit 5 UDRIE- DATA REGISTER EMPTY INTERRUPT ENABLE
powyższe bity dotyczą przerwań

•Bit 4 RXEN- RECEIVER ENABLE
•Bit 3 TXEN- TRANSMITER ENABLE
on/off odbiornika, domyślnie wyłączone
on/off nadajnika, domyślnie wyłączone

Linie nadajnika i odbiornika są na porcie PB0 i PB1- normalnie te linie funkcjonują jako zwykłe linie wejściowo-wyjściowe. Jeśli chcemy uruchomić nadajnik musimy aktywować powyższe bity (ustawić 1).

•Bit 2 UCSZ2- CHARAKTER SIZE
Razem z pozostałymi bitami UCSZ1 i UCSZ0 z rejestru UCSRC będzie odpowiadał za liczbę bitów
•Bit 1 RXB8- RECEIVER CATA BIT8
•Bit 0 TXB8-TRANSMITER DATA BIT 8
Powyższe 2 bity wykorzystywane są przy transmisji 9 bitowej w ramce

USART Control and Status Register C-

Bit	7	6	5	4	3	2	1	0	
	URSEL	UMSEL	UPM1	UPM0	USBS	UCSZ1	UCSZ0	UCPOL	UCSRC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	1	0	0	0	0	1	1	0	

•Bit 7 URSEL - REGISTER SELECT
Ten bit związany jest, że pod tym samym adresem umieszczono dwa rejestry-UCSRC i UBRRH. Zastosowano takie rozwiązanie prawdopodobnie z braku przestrzeni adresowej. Wybór, do którego rejestru chcemy się odwołać odbywa się właśnie przez ten bit nr 7

•Bit 6 UMSEL - MODE SELECT
Ustala tryb transmisji. Domyślnie UMSEL==0 →transmisja asynchroniczna

•Bit 5:4 UPM1:0 PARITY MODE
Bity kontroli parzystości. Domyślnie UPM1:0==0 → wyłączona kontrola parzystości

•Bit 3 USBS - STOP BIT SELECT
Ustala liczbę bitów stopu. Domyślnie USBS==0 → 1 BIT STOPU

•Bit 2 :1 UCSZ1:0 CHARAKTER SIZE
Razem z UCSZ2 w rejestrze UCSRB. Możemy wysyłać od 5 do 9 bitów w ramce.
Domyślne dla 8 bitów: UCSZ2==0 UCSZ1==1 UCSZ0==1

•Bit 0 UCPOL CLOCK POLARITY
Bit związany z transmisją synchroniczną(polaryzacją zegara).

UART:
stan jałowy 1, potem bit startu 0 i potem bity stopu 1 - generalnie powiedzmy nadawca co 16 cykli zegara zmienia stan, tzn. przesyła kolejny bit i odbiorca musi się do tego zastosować, zazwyczaj robi się to po prostu w kodzie, można zrobić mechanizm autonegocjacji, jednak nie jest to często stosowane.